

Tech Stack Summary

TrackServ is a **multi-frontend civic service platform** with three React applications connected to one Supabase backend.

Frontend

Citizen portal

Location: TrackServ

- React 19.2
- Create React App
- react-scripts 5.0.1
- React Router 7.18
- JavaScript and JSX
- CSS
- lucide-react for icons
- react-leaflet and Leaflet for maps
- Recharts for charts
- Supabase JavaScript client
- Testing Library and Jest through Create React App
- GitHub Pages deployment

Main entry point: App.js:1-98

Purpose:

- Citizen registration and login
- Municipal issue reporting
- Image uploads
- Report tracking
- Voting
- Community posts and comments
- Service listings and advertisements
- Citizen profiles

Administration portal

Location: admin-dashboard

- React 19.2
- Create React App
- react-scripts 5.0.1
- React Router 7.18
- HashRouter
- JavaScript and JSX
- CSS
- lucide-react
- Leaflet and React Leaflet
- Recharts
- Supabase JavaScript client
- GitHub Pages deployment

Main entry point: App.js:1-88

Purpose:

- Review reports
- Change report statuses
- Assign staff
- Schedule events
- Analyze report data
- View report locations
- Manage operational workflows

Staff portal

Location: TrackServers/servers

- React 19.2
- Vite 8
- @vitejs/plugin-react

- React Compiler Babel preset
- React Router 6.30
- JavaScript and JSX
- ESLint 10
- lucide-react
- Leaflet and React Leaflet
- Recharts
- Supabase JavaScript client
- GitHub Pages deployment

Main entry point: App.jsx:1-18

Purpose:

- Show assigned reports
- Display issue locations
- Calculate routes
- Update report progress
- Upload completion evidence
- Submit resolution notes
- View staff history and profile information

Backend

There is no custom Node.js, Express, Laravel, Django, or Java backend in the workspace.

The backend is **Supabase**, consisting of:

Database

- Hosted PostgreSQL
- Relational schema with UUID primary keys
- Foreign keys between profiles, reports, categories, events, assignments, groups, and resolutions
- Check constraints for roles, statuses, severities, payment states, and ratings
- Row Level Security enabled, according to your confirmation

The main domain tables are:

- profiles
- reports
- categories
- report_images
- report_votes
- assignments
- groups
- group_members
- events
- issue_resolutions
- community_posts
- comments
- likes
- saved_posts
- service_categories
- service_listings
- listing_reviews
- ad_payments

Authentication

Supabase Auth handles:

- Email/password registration
- Login
- Logout
- Password reset
- Session management

The profiles.role column provides application roles:

- citizen

- staff
- moderator
- admin

Each frontend creates a Supabase client in its own source tree, for example `supabaseClient.js:1-6`.

API/data access

The applications access PostgreSQL through the Supabase JavaScript SDK:

- `supabase.from(...).select()`
- `insert()`
- `update()`
- `delete()`
- `supabase.auth`
- `supabase.storage`
- `supabase.channel()`

Supabase automatically provides the REST/PostgREST API layer. There is no separately maintained API server.

File storage

Supabase Storage is used for:

- Citizen report images
- Profile pictures
- Community images
- Advertisement images
- Staff resolution evidence

The main buckets referenced in code include `images` and `Profile`.

Realtime

The admin application subscribes to PostgreSQL changes for reports in `ReportsPage.jsx:431-444`.

This allows administrative report lists to react to database changes without a full page refresh.

External Services

The system also uses:

- OpenStreetMap tiles for maps
- Nominatim for geocoding and address lookup
- OSRM for staff driving-route calculations
- GitHub Pages for hosting
- GitHub Actions for continuous deployment

The OSRM integration is visible in `Dashboard.jsx:347-388`.

Hosting and Deployment

All three applications are deployed as static frontends to GitHub Pages:

- Citizen portal: TrackServ
- Admin portal: admin-dashboard
- Staff portal: /TrackStaff/

Deployment is handled with GitHub Actions and `peaceiris/actions-gh-pages`.

The staff application configures its Vite base path in `vite.config.js:1-11`.

Architecture Style

The system is currently a:

Three-frontend, shared-backend, role-based SaaS-style civic platform

The applications are separate deployments but share:

- One Supabase project
- One authentication system
- One PostgreSQL database
- One role model
- Shared report lifecycle
- Shared storage resources

The operational flow is:

1. Citizen submits a report.
2. Report is stored in PostgreSQL.

3. Admin reviews and assigns it.
4. Staff sees the assignment.
5. Staff updates the report and submits resolution evidence.
6. Citizen and administrators see the resulting status.

What You Do Not Currently Have

The repository does not show:

- A custom backend server
- REST or GraphQL controllers
- A separate API gateway
- A backend ORM such as Prisma or Sequelize
- TypeScript
- A monorepo workspace
- Shared component packages
- Automated end-to-end testing
- Database migration files
- Supabase Edge Functions in the attached projects

Main Stack Definition

A concise technical description of your system would be:

TrackServ is a JavaScript and React-based civic issue-management platform composed of three independently deployed React frontends. The citizen and administration portals use Create React App, while the staff portal uses Vite. All applications use React Router, Supabase Auth, Supabase Storage, PostgreSQL through Supabase's PostgREST API, Row Level Security, and Supabase Realtime. Mapping uses Leaflet, React Leaflet, OpenStreetMap, Nominatim, and OSRM. Charts use Recharts, icons use Lucide React, and deployments run through GitHub Actions to GitHub Pages.